

Functional Requirements Specification

Stakeholders:

- Customers: End users who browse movies, book tickets, and make payments.
- Cinema Managers: Oversee movie schedules, seat availability, and revenue tracking.
- System Administrators: Support the platform, manage user accounts, and troubleshoot issues.
- Sponsors/Owners: Fund the system and check performance metrics.
- Developers: Build and sustain the system's features and infrastructure.

Primary Actors:

- Customer: This actor can browse movies and showtimes, select seats and book tickets, and make payments and receive confirmation messages.
- Cinema Manager: The cinema manager can add/update movie listings and showtimes, as well as monitor bookings and manage seat availability.

Secondary Actors:

- System Administrators: This can manage user accounts and system configurations, as well as handle escalated issues and keep data integrity.
- Payment Gateway: This can process transactions securely via credit card or digital wallet.

Use Cases:

Customer (total: 14)

- Register/Login (2): To create an account or log in to access booking features.
- Browse Movies (2): To view current and upcoming movies with showtimes and trailers.
- Select Showtime (2): To choose a preferred date and time for a movie.
- Choose Seats (2): To pick available seats from a visual layout.
- Make Payment (2): To pay for selected tickets via credit card or wallet.
- Receive Confirmation (2): To get booking details via email or SMS.
- Cancel Booking (2): To cancel a ticket before showtime and request refund.

Cinema Manager (total: 10)

- Add Movie (2): To upload movie details including title, genre, duration, and poster.
- Schedule Showtimes (2): To dates, times, and screen assignments.

- Update Availability (2): To change seat layout or mark seats as unavailable.
- View Bookings (2): To track ticket sales and occupancy rates.
- Remove Movie (2): To remove outdated or canceled listings.

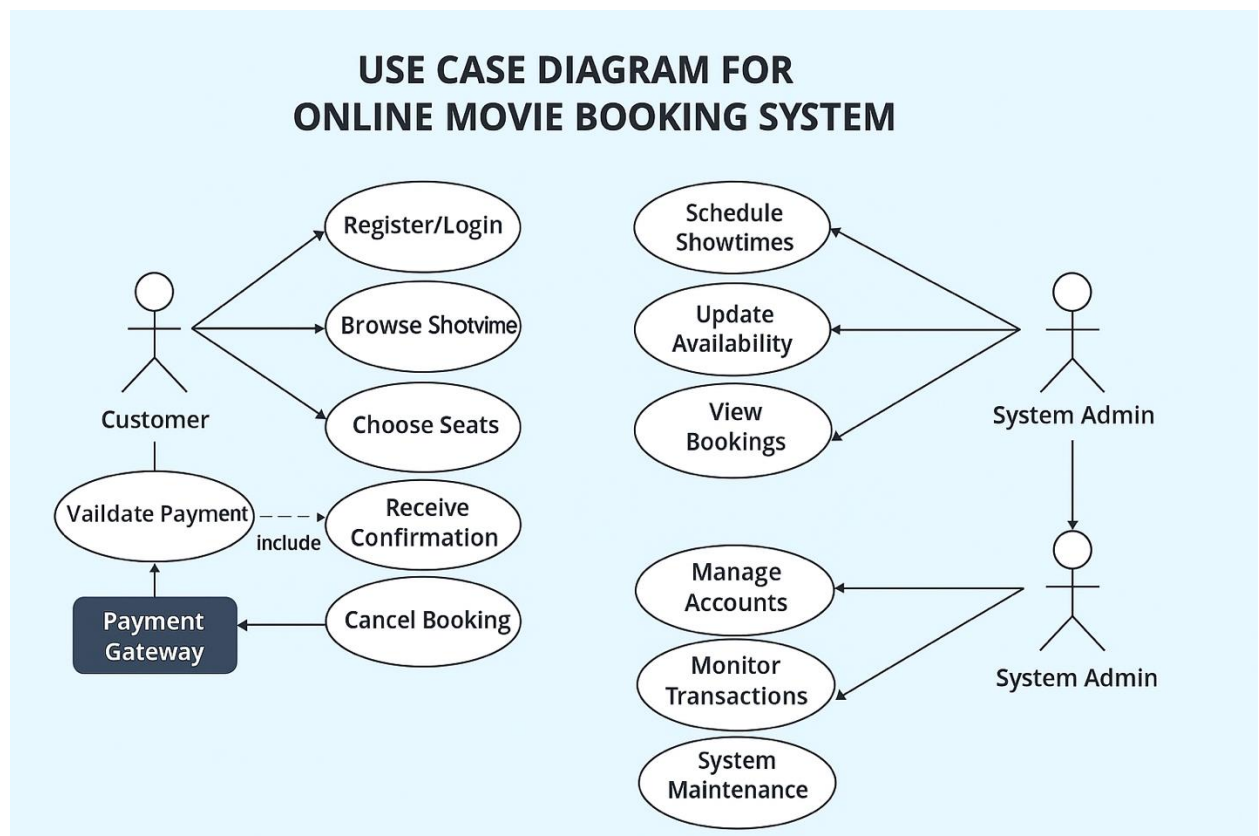
System Admin (total: 8)

- Manage Accounts (2): To add, remove, or update user profiles.
- Monitor Transactions (2): To review payment logs and resolve disputes.
- System Maintenance (2): To perform updates and fix bugs.
- Generate Reports (2): To create summaries of revenue, traffic, and usage.

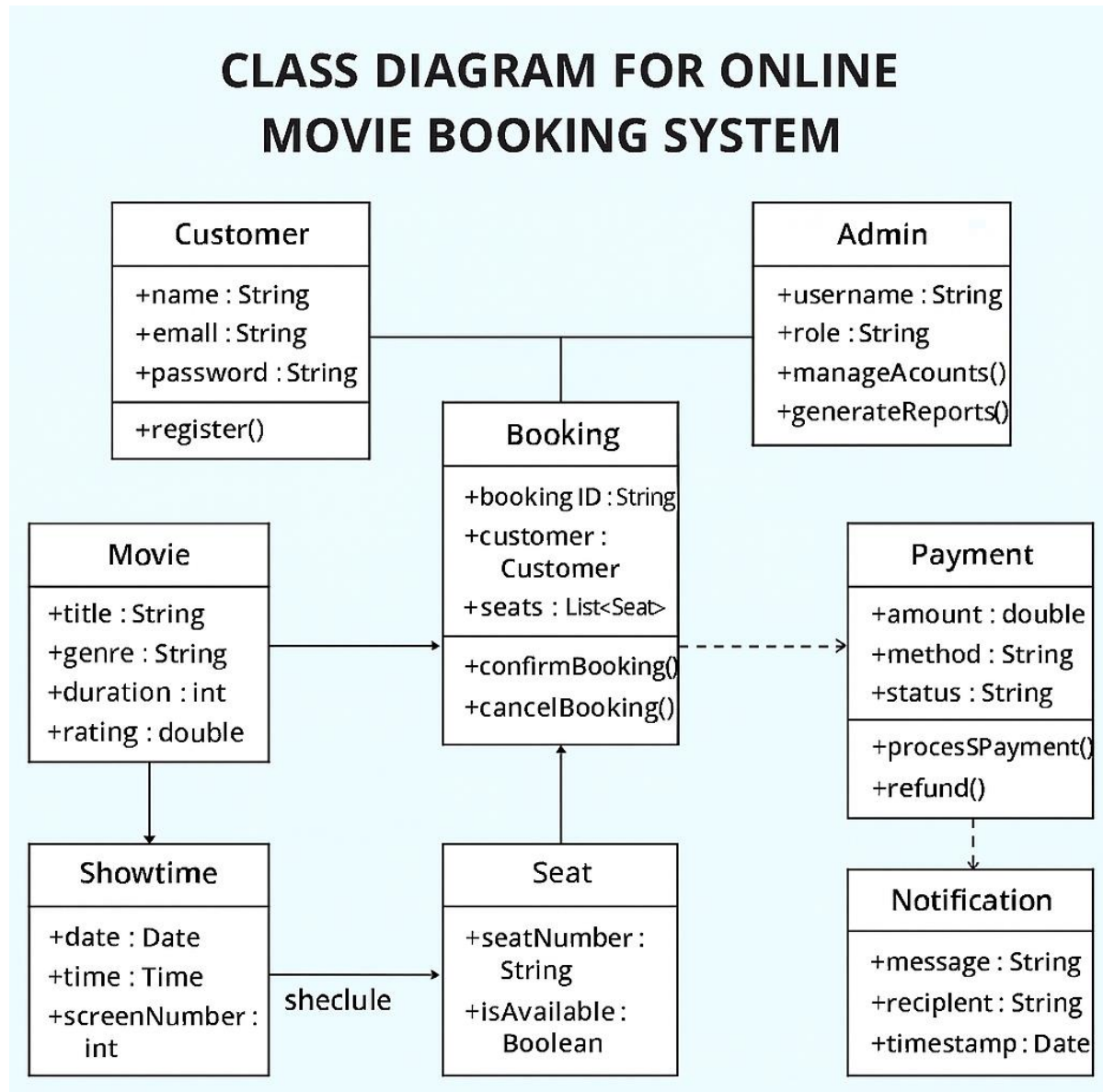
System (total: 8)

- Validate Payment (2): To confirm transaction success before issuing ticket.
- Send Notifications (2): To deliver booking confirmations and reminders.
- Update Seat Map (2): To reflect real-time seat availability.
- Log Activity (2): To record user actions for analytics and security.

Use Case Diagram:



Class Diagram:



Data Types and Operation Signatures:

-Customer-

- + name : String
- + email : String
- + password : String
- + register() : void
- + login() : boolean
- + bookTicket(movie : Movie, showtime : Showtime, seats : List<Seat>) : Booking

-Movie-

- + title : String
- + genre : String
- + duration : int
- + rating : double
- + addMovie() : void
- + updateMovie() : void

-Showtime-

- + date : Date
- + time : Time
- + screenNumber : int
- + scheduleShowtime(movie : Movie) : void
- + updateTime(newTime : Time) : void

-Seat-

- + seatNumber : String
- + isAvailable : Boolean
- + reserve() : void

+ release() : void

-Booking-

+ bookingID : String

+ customer : Customer

+ seats : List<Seat>

+ status : String

+ confirmBooking() : void

+ cancelBooking() : void

-Payment-

+ amount : double

+ method : String

+ status : String

+ processPayment() : boolean

+ refund() : boolean

-Admin-

+ username : String

+ role : String

+ manageAccounts() : void

+ generateReports() : Report

-Notification-

+ message : String

+ recipient : String

- + timestamp : Date
- + sendEmail() : void
- + sendSMS() : void

Relationship Summary:

Customer → Booking (association)

Booking → Movie, Showtime, Seat, Payment (association)

Admin → Movie, Showtime (association)

Notification → Customer (association)

Seat → Booking (association)

Traceability Matrix:

System Requirements

No.	Priority Weight (1-5)	Description
REQ1	1	Users can browse available movies and showtimes
REQ2	5	Users can book movie tickets online
REQ3	4	Users can cancel or modify bookings
REQ4	5	Admin and Agents can log in/out from the system
REQ5	3	Admin can add, update, or delete movie listings
REQ6	1	System sends booking confirmation via email
REQ7	2	Users can create and manage personal profiles
REQ8	1	System provides search and filter options for movies
REQ9	1	System logs user activity for audit and analytics

Use Cases

No.	Description
UC1	User browses movies and showtimes
UC2	User books tickets for a selected movie
UC3	User cancels or modifies an existing booking
UC4	Admin/Agent logs into the system
UC5	Admin manages movie listings (add/update/delete)
UC6	System sends booking confirmation to user
UC7	User creates or updates personal profile
UC8	User searches and filters movies based on genre, time, or location

Req't	PW	UC1	UC2	UC3	UC4	UC5	UC6	UC7	UC8
REQ1	5	X							
REQ2	2		X						
REQ3	5			X					
REQ4	4				X				
REQ5	2					X			
REQ6	1						X		
REQ7	1							X	
REQ8	1								X
REQ9	1								X
Max PW		5	2	5	4	2	1	1	1
Total PW		5	2	5	4	2	1	1	2

System Architecture and System Design

Architectural Styles

The Online Movie Booking System uses a client-server architectural style. The client is a web browser that communicates with the server by sending requests and receiving responses. The client does not need to understand the internal workings of the server to interact with it. Instead, the client requests web pages and booking actions, and the web server responds accordingly.

The server is a computer that provides services to the client. I am using a three-tier architecture to build the Online Movie Booking System:

- The presentation layer runs on the client and displays the user interface using HTML, CSS, and JavaScript.
- The application layer runs on the server and handles business logic using a backend framework such as Node.js or PHP.
- The data layer uses a relational database (e.g., MongoDB) to store persistent data such as movies, showtimes, users, and bookings.

The system runs across multiple machines. The client accesses the system through a browser, while the server hosts the application and database. The server handles requests from the client, processes them, and interacts with the database to retrieve or store data.

Identifying Subsystems

The UML package diagram shown below portrays the subsystems of the Online Movie Booking System. The system is designed with a client-server architecture, which includes an application layer and a database storage layer.

The application has a User Interface package that displays information to the user and allows interaction through buttons and forms. This interface connects to packages such as User Management, Movie Catalog, Showtime Scheduler, Seat Selection, Booking Processor, and Payment Gateway.

Each package handles specific functionality:

- User Management manages login, registration, and profile updates.
- Movie Catalog displays available movies and metadata.
- Showtime Scheduler manages showtimes and availability.
- Seat Selection allows users to choose seats from a visual map.
- Booking Processor validates and confirms bookings.
- Payment Gateway processes payments securely.

These packages interact with the Database Storage Layer, which stores persistent data. The application sends SQL queries to the database, which responds with requested data or updates records. The results are then displayed back to the user on the web page.